TechMove logistics design

By: Keshav Haripaul

Table of contents

Introduction and summary

The company is currently operating using a legacy system that comprises of disjointed spreadsheets, manual phone calls and emails, which has resulted in data inconsistencies such as lost invoices, and inefficient workflow management. These limitations hinder the organisation's ability to scale and maintain compliance with contractual obligations. This report presents the proposed design for the GLMS, an enterprise-grade solution aimed at centralising contract management, streamlining service request processing, and integrating financial operations. The scope of this project includes the design of a scalable web platform that can handle contracts, clients and service request while also having high availability and system reliability.

The intended audience for this report is the stakeholders that hold a share in the business and the current board of directors, as they require information on how the system will operate and how the system will sort out the issues of the legacy system and how the business will grow in the future due to this system change. The report outlines the architectural decisions, design patterns, and scalability strategies that will be implemented to address the organisation's challenges.

The architectural vision for the GLMS is based on a modern, service-oriented design using the TOGAF framework to align business and IT objectives. The system will utilise ASP.NET Core technologies, RESTful APIs, and a layered architecture separating presentation, business logic, and data access. Additionally, the solution incorporates

design patterns such as Factory, Strategy, and Observer to ensure modularity and flexibility.

To support future expansion, the system is designed with scalability in mind, incorporating horizontal scaling, load balancing, and caching mechanisms such as Redis. This ensures that the new system can handle the increase in workload due to the addition of any new contracts while also not failing in performance.

Enterprise framework strategy

For this project, the TOGAF be used to structure the development of the GLMS. This design will focus on an automative service processing solution and will integrate the financial operations to be digital. TOGAF provides a reliable approach to designing enterprise systems using the ADM method. For this project, the first four phases (A–D) are applied. The proposed system will improve operational efficiency, enhance data consistency, and provide a future-ready platform for TechMove Logistics.

Phase A – Architecture Vision

The main objective of Phase A is to centralise contract management, and track service requests. The system will be developed using C# to support enterprise-level logistic operations.

Phase B – Business architecture

It will state the business processes required by the system. The key processes of this phase will include creating and monitoring the client contracts, being able to raise requests linked to the contracts, validating if a contract is active or expired and processing financial data. Actors include administrators, logistics managers and clients.

Phase C – Information Systems Architecture

This phase describes and uses both architecture types, data and application.

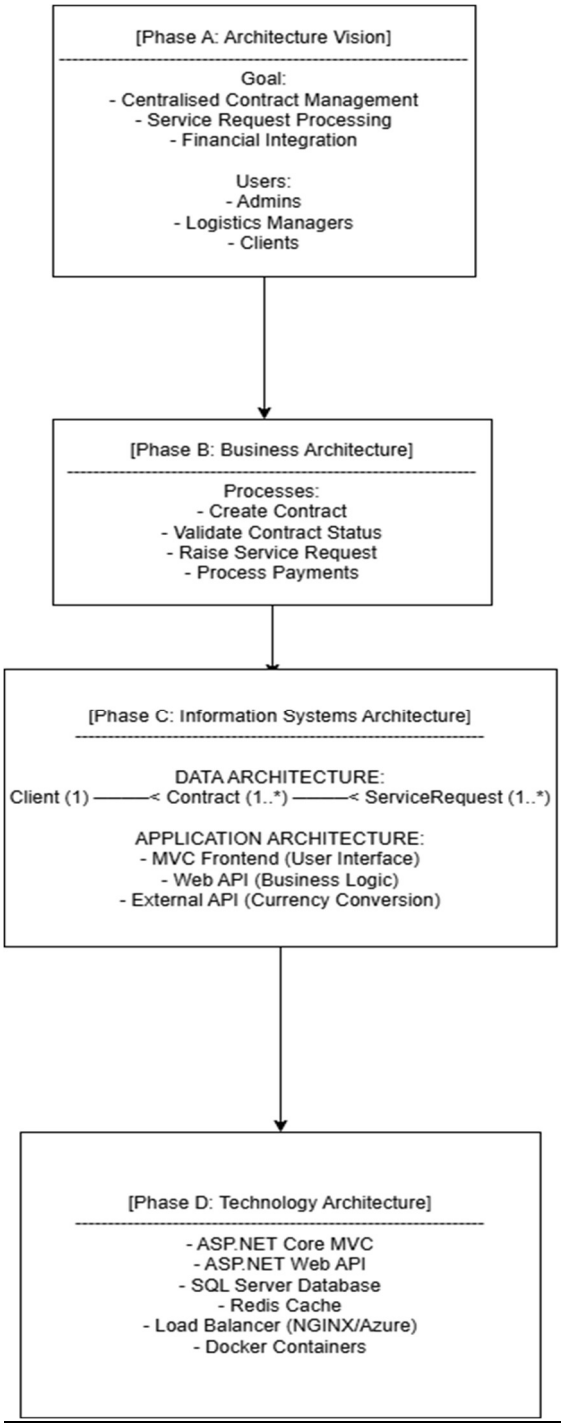
Data architecture. Important data entries include Client, Contract, ServiceRequest , these entities will store and manage information within the system. The relationship in the system is that one client can have many contracts, and one contract can have many serviceRequests.

Application architecture: - MVC Web Application (Frontend), Web API (Backend), API integration and a sql server database.

Phase D – Technology architecture

The technology architecture defines the technical tool the system is using, possible technologies include C#, .NET Framework / .NET Core, SQL server etc. These technologies allow the system to support scalability and integration.

Enterprise application



Design patterns

The system will implement three of the Gang of Four (GoF) design patterns.

The selected patterns include:

1. Factory Method Pattern
2. Strategy Pattern
3. Observer Pattern

These patterns improve **code maintainability, flexibility, and scalability.**

Selection and justification

1. The Factory Method pattern is used to create objects without specifying the exact class that will be instantiated.

In the GLMS system, contracts can be decided using different methods such as:

- Standard Contract
- Premium Contract
- International Contract

The factory method determines which contract type should be created based on input parameters or the rules of the business.

Benefits

- Reduces tight coupling, where the system would otherwise depend on concrete classes.
- Simplifies object creation
- Makes the system easier to extend.

Additionally, the factory pattern supports the open/closed principle where new contract types can be introduced without changing the existing code. This is good for TechMove, as logistics contracts may evolve over time due to business changes or regulatory changes.

Strategy pattern

The Strategy pattern allows different algorithms to be selected dynamically at runtime.

In GLMS, different cost strategies may exist such as:

- Standardcoststrategy
- Expresscoststrategy
- Internationalcoststrategy

Each strategy calculates delivery cost differently.

Benefits

- Flexible algorithm selection
- Reduces conditional statements, by eliminating the need for multiple if-else blocks and makes the system more modular.
- Easier system maintenance
- It also ensures that new pricing strategies can be added without modifying existing classes, improving maintainability.

For TechMove this pattern is critical as pricing models may change frequently due to currency changes, logistics complexity or new service offerings.

Observer pattern

The Observer pattern allows objects to receive automatic updates when the state of another object changes, such as the contracts.

In GLMS, customers need to receive contract updates such as:

- Contract Active
- Contract On Hold
- Contract expired

Observers such as customers are notified whenever the contract status changes.

Benefits

- Real-time notifications
- Loose coupling, as the subject does not need to know the details of the observers.
- Event-driven architecture is essential for real-time updates in enterprise systems.

For TechMove this ensures that important events such as contract expiry or service request updates are communicated as soon it is known, improving operational efficiency and compliance.

Logic and structure

Factory pattern relationships

The factory method pattern will be implemented using interface-based dependency and inheritance. The IContract interface defines the common behaviour for all the contract types. There will concrete classes called StandardContract and PremiumContract that will implement this interface. The ContractFactory class will depend on the IContract interface rather than concrete implementations.

This will create a dependency relationship, where the factory will produce objects through abstraction. The factory enscapsulates object creation logic, ensuring that the rest of the system will not be tightly coupled to specific contract classes.

Strategy pattern relationships

The strategy pattern will be implemented using composition and polymorphism. The IServiceCostStrategy interface will define the cost calculation method. There will be concrete strategies such as StandardCostStrategy, ExpressCostStrategy, and InternationalCostStrategy that will implement this interface. The ServiceRequest class contains a reference to IServiceRequest.

This will form a composition relationship where the ServiceRequest object uses a strategy object to perform the cost calculations. Due to the strategy being referenced through an interface, different algorithms can be substituted dynamically at runtime without modifying the ServiceRequest class.

Observe pattern relationship

The observer pattern will be implemented using a one-to-many relationship. The Contract class acts as the Subject and maintains a list of IObserver objects. The IObserver interface defines the Update() method. There will be concrete observers such as NotificationService and LoggingService implement this interface.

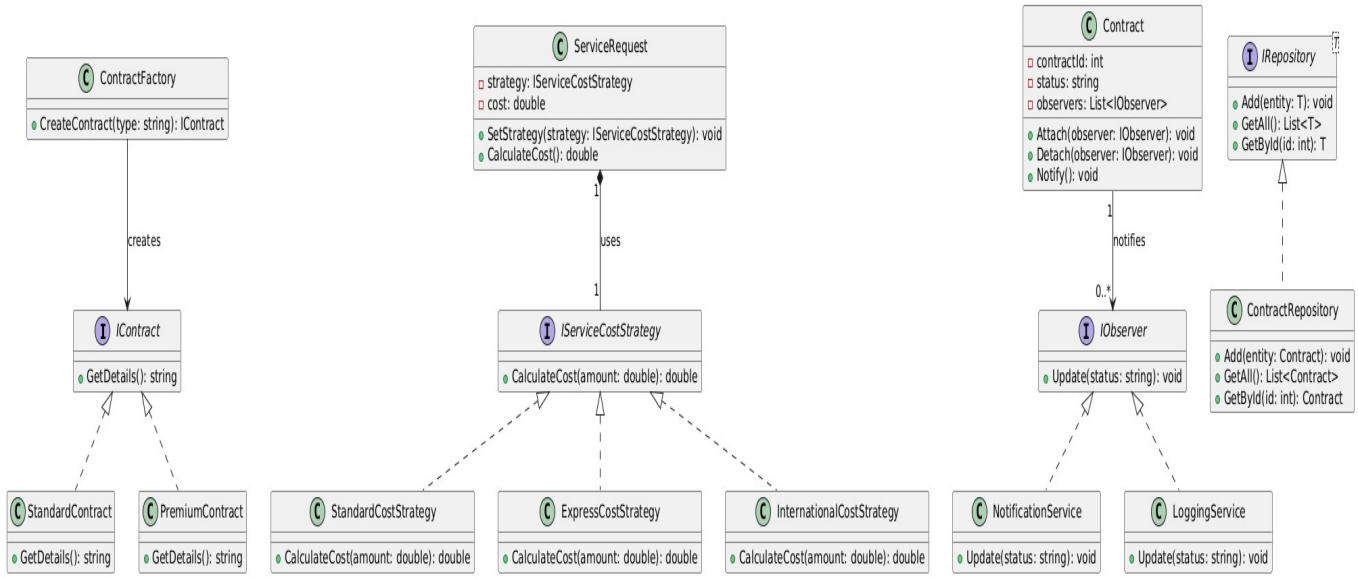
The relationship has a cardinality of 1 to 0...*, meaning one contract can notify many observers. This ensures that multiple system components can react to changes in the contract state without being tightly coupled to the contract class.

Dependency injection is used across all the design patterns to ensure loose coupling and improve maintainability. There is interface-based injection where all components will depend on an interface rather than concrete classes. These include IContract for contract creation, IServiceCostStrategy for cost calculation, and IObserver for notification handling. This allows implementations to be changed or extended without changing any of the dependent classes.

There is an event trigger in the observer pattern. An event will be triggered when the state of a contract changes, such as a contract is expired or is active. When the status change occurs, the Contract object will update its internal status attribute. Then the Notify() method is invoked. All registered observers will then be iterated through. Each observer then executes its Update() method. Different observers perform different actions. The NotificationService observer sends alerts to users or system dashboards and the LoggingService observer records the event for auditing and compliance tracking.

The advantages of having event-driven design is that it enables real-time system updates, it reduces direct dependencies between components, allows new observers to be added easily and supports scalable and reactive system design. The implementation of these design patterns through the use of proper relationships, dependency injection techniques and event driven behaviour ensures that the system has high flexibility through interchangeable components, reduced coupling between classes and improved maintainability and scalability.

UML Class diagram



Scalability & Non-functional Requirement Strategy

Availability is a critical non-functional requirement for any system. It refers to the ability of the system to be operational, accessible, and responsive to users always. For TechMove logistics, which operates across multiple regions, system downtime could be detrimental for the business. To ensure reliability, the GLMS must reach a minimum availability target of 99.9 % uptime. This level of availability ensures that the system is accessible continuously, with only minimal unplanned or planned downtime.

The ability of the new system to communicate and exchange data with other systems, both internal and external is referred to as Interoperability. In a modern enterprise environment, systems never operate in isolation, and seamless integration is essential for efficiency. The GLMS must be able to integrate with many internal and external systems. One key external integration is with external currency exchange APIs, which are required to convert international currencies to local currency.

To handle a 50 000 increase in contracts, the GLMS must implement a scalability strategy that the system architecture can handle the load. Different strategies will be discussed such as horizontal scaling, load balancing, and caching.

For horizontal scaling the new system will use a horizontal scaling approach where multiple instances of the system are deployed across different locations such as containers or servers. Where each instance runs the same application logic, new instances can be added dynamically based on demand and containerisation enables rapid scaling. This ensure that the increased user requests and contract processing is distributed evenly across multiple nodes.

A load balancer is used to distribute incoming traffic such as the 50000 contracts across multiple application instances, ensuring no single server becomes a bottleneck. The load balancing strategy uses a round robin algorithm where requests are distributed evenly across all available servers, a least connections algorithm where requests are sent to the sever with the least requests and health checks where the load balancer continuously monitors the server health and removes failed nodes automatically. Tools such as Azure load balancer will be used. This ensure high availability, improved response times and fault tolerance for the system.

To reduce the load of the 50 000 contracts on the database and improve the performance, the new system will implement a distributed caching layer. This will be done using Redis. There will be 3 levels. The first level is application-level cache where frequently accessed data such as active contracts or client details is stored in Redis. This reduces the repeated database queries. The second level is query result cache where results of expensive queries such as filtering contracts by date or status are cached. This will improve performance for repeated searches. The last level is Session cache where user session data is stored in Redis. This ensures consistency across multiple servers in a load balanced environment.

The cache workflow will look like this:

User Request → Check Redis Cache → (Hit) Return Data

→ (Miss) Query Database → Store in Cache → Return Data

The benefits of Redis caching is that it uses in memory storage which means the response times are extremely fast, it reduces database load significantly, it supports distributed environments and it improves user experience and scalability.

By using different scalability strategies, the new system will handle the increase in contracts by 50 000. This architecture ensures that the system has high performance, reduced latency and system stability even under extreme workloads.

References

Gamma, E., Helm, R., Johnson, R. and Vlissides, J., 1994. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.

The Open Group, 2018. *TOGAF® Standard, Version 9.2*. Van Haren Publishing.

Sommerville, I., 2016. *Software Engineering*. 10th ed. Harlow: Pearson.

Richards, M. and Ford, N., 2020. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol: O'Reilly Media.

Fowler, M., 2002. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Microsoft, 2023. *ASP.NET Core documentation*. [online] Available at: https://learn.microsoft.com/en-us/aspnet/core/ [Accessed 24 March 2026]

Redis Ltd., 2023. *What is Redis?* [online] Available at: https://redis.io/docs/ [Accessed 24 March 2026].

Amazon Web Services, 2023. *Elastic Load Balancing*. [online] Available at: https://aws.amazon.com/elasticloadbalancing/ [Accessed 24 March 2026]

Newman, S., 2015. *Building Microservices: Designing Fine-Grained Systems*. Sebastopol: O'Reilly Media.